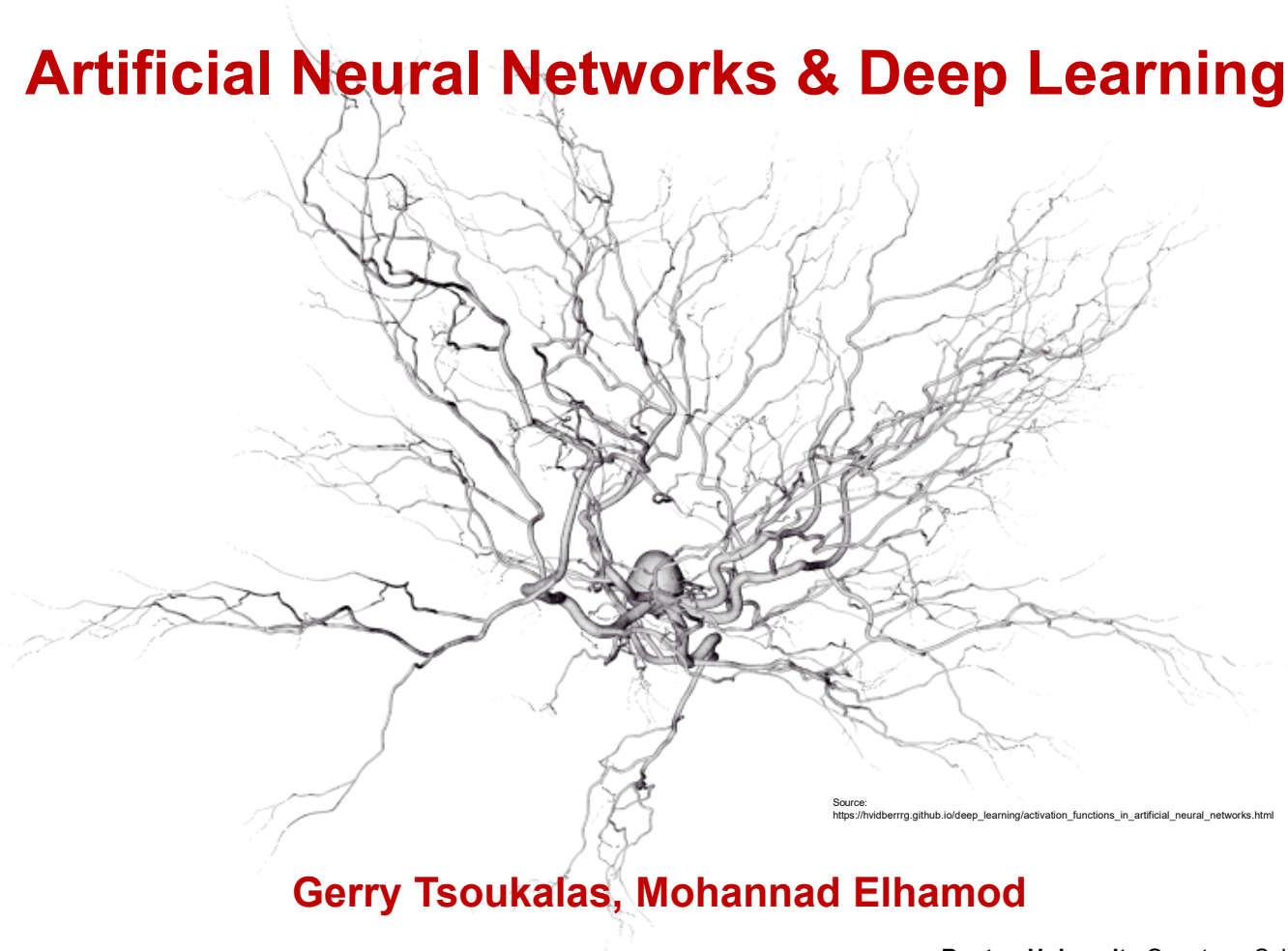
Artificial Neural Networks & Deep Learning



Source:
https://hvidberrrg.github.io/deep_learning/activation_functions_in_artificial_neural_networks.html

Gerry Tsoukalas, Mohannad Elhamod

Boston University Questrom School of Business

Before we get started…

- Midterm #2 focuses on decision trees, logistic regression and neural networks. However, subjects such as visualization and evaluation are fair game.

Introduction

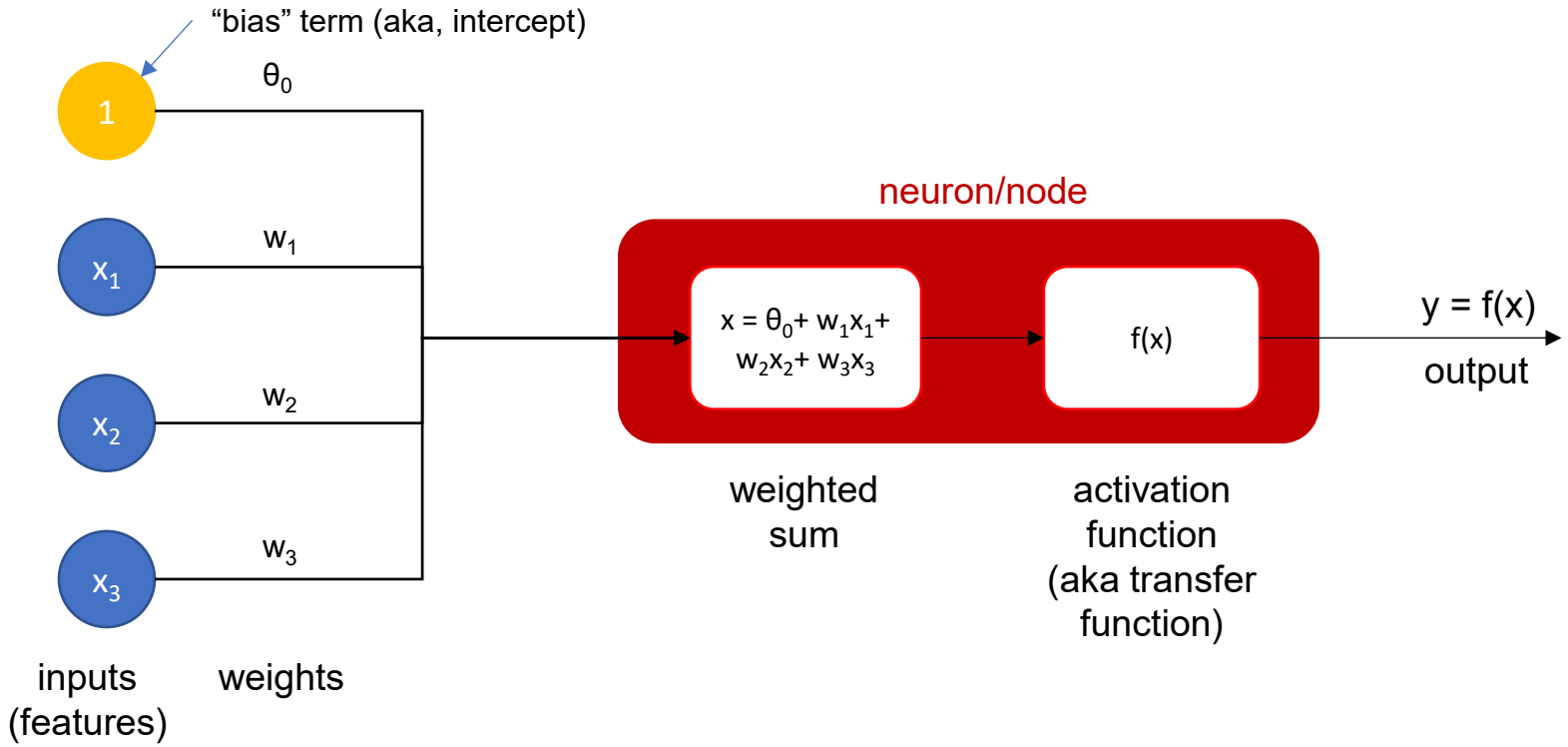
- What are (artificial) neural networks?
  - flexible black-box methods for classification, regression, and feature extraction (PCA)
  - building blocks for “deep learning”
    - Often used for complex voice and image recognition tasks, (e.g. amazon alexa, translation, face unlock…)
    - Successfully deployed in many business settings such as stock price predictions, credit card fraud detection, self-driving cars, etc.
- Why are they called neural networks?
  - inspired by how neurons and their connections, work in the brain
  - try to mimic the way we believe the human brain “learns” some tasks
- Some of their strengths…
  - high predictive performance
  - can capture complicated, “non-linear” relationships between variables
- Some of their issues…
  - infinitely large design space, i.e., the analyst has to set many many many different parameters
  - prone to overfitting

How does a (real) neuron work? – uses threshold firing

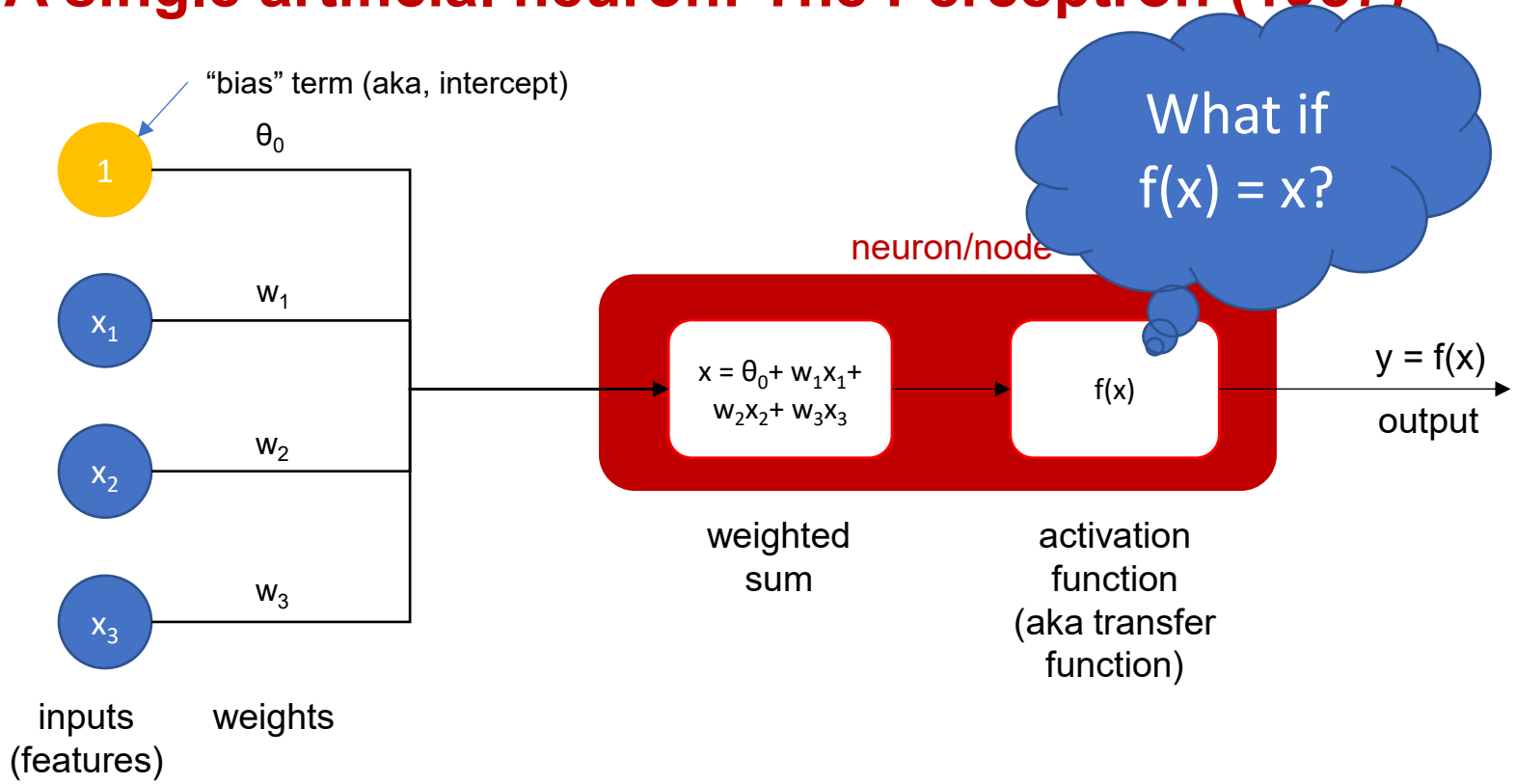


- it receives weighted input from various other neurons
- these signals are combined and processed by the receiving neuron
- if the strength of the resulting signal is above a certain threshold, the neuron fires an output signal to neighboring neurons
- 86 bn neurons
- 100 tr connections

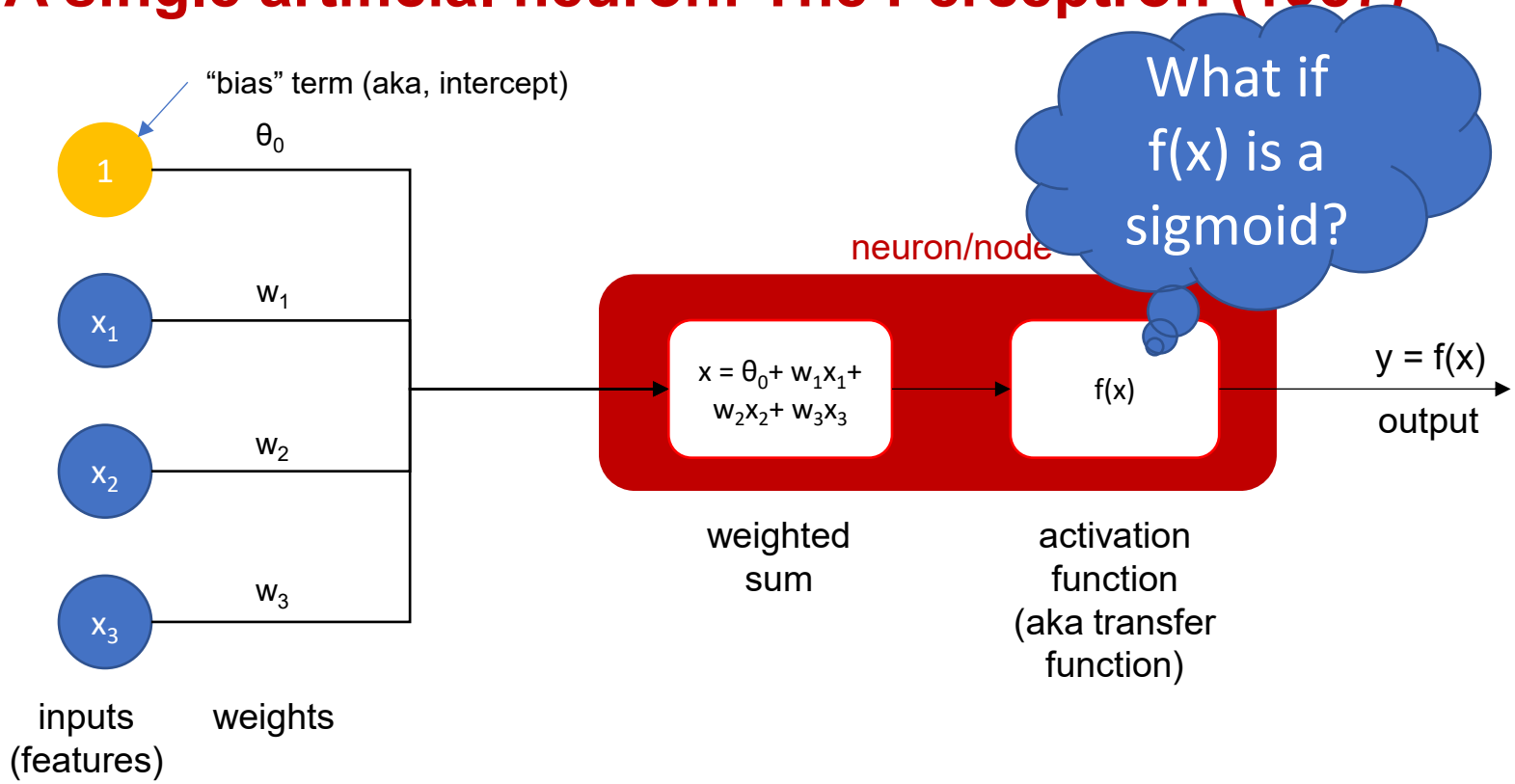
A single artificial neuron: The Perceptron (1957)

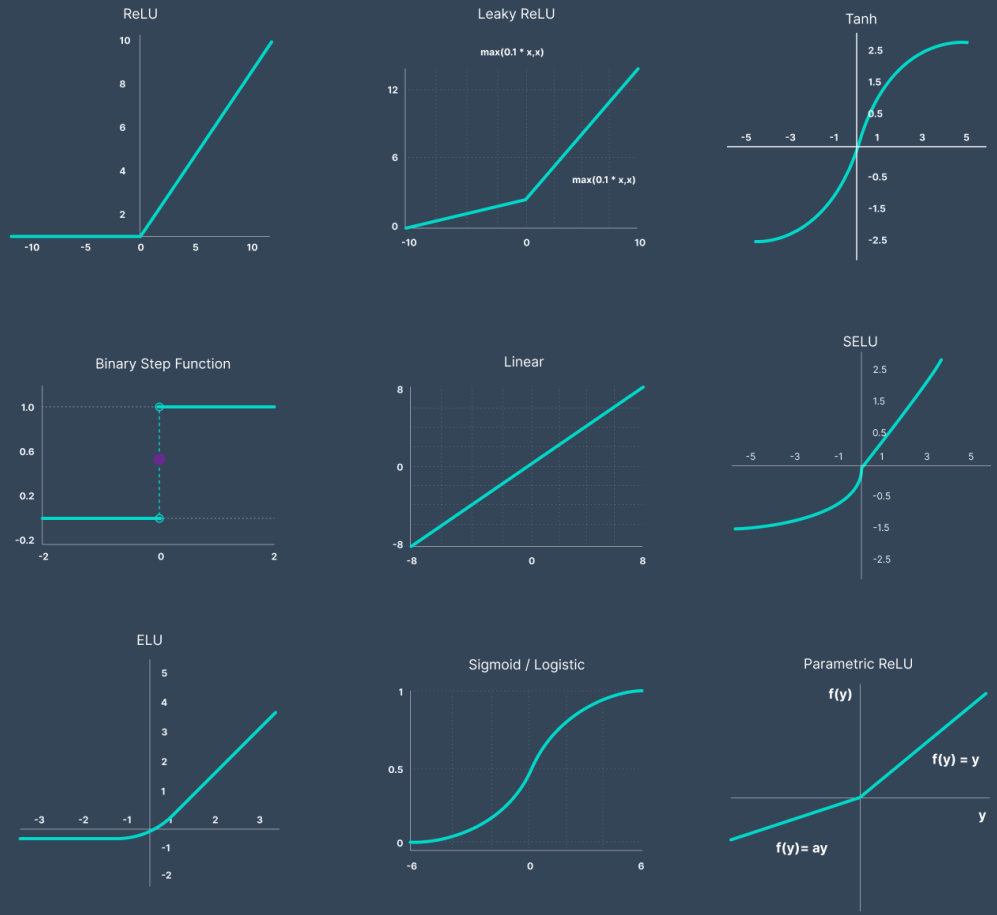


A single artificial neuron: The Perceptron (1957)



A single artificial neuron: The Perceptron (1957)

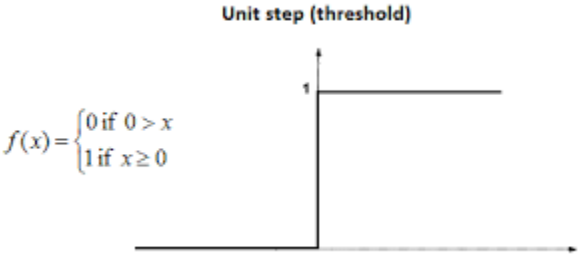


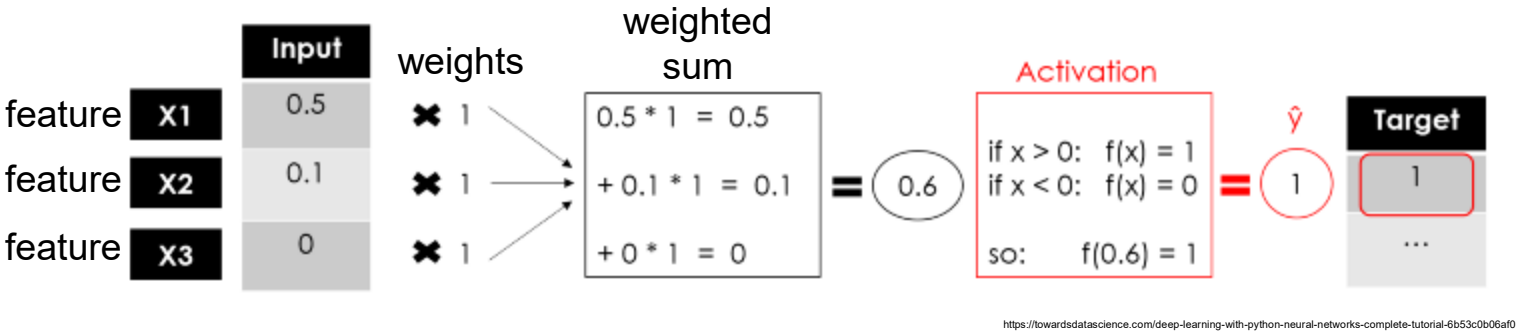


- Examples of “activation functions”
- sigmoid, ste, ReLU, and tanh are the most common ones
- Notice: the design space is already infinite here. There are countless functions.

Simple Example of a Perceptron

- Assume 3 input features, no bias, and "step" activation function f(x)
- Computations for a single row of the data (repeat for all rows):





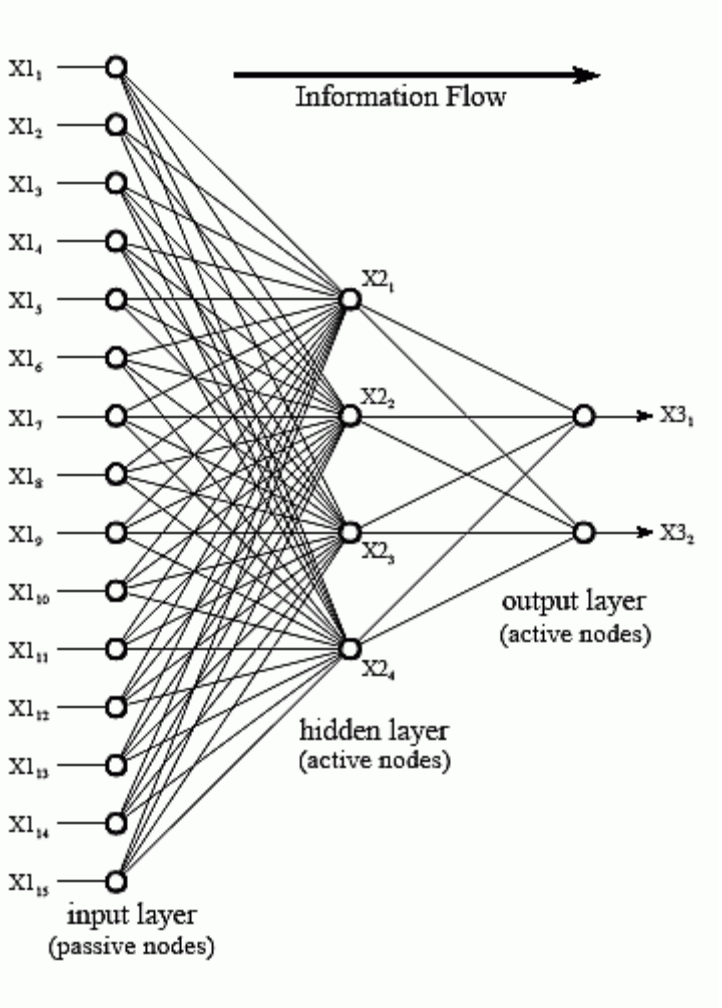
https://towardsdatascience.com/deep-learning-with-python-neural-networks-complete-tutorial-6b53c0b06af0

- the goal is to find the optimal weights so as to reduce the error between output y and target
- Algorithms (e.g., gradient descent) are used to find the optimal weights

Neural Networks

- A neural network consists of a *layered*, *feedforward*, *fully connected* network of perceptrons
  - Multiple layers: Input, Hidden, Output
  - Nodes (neurons)
  - "**Deep learning**" refers to ~3+ hidden layers

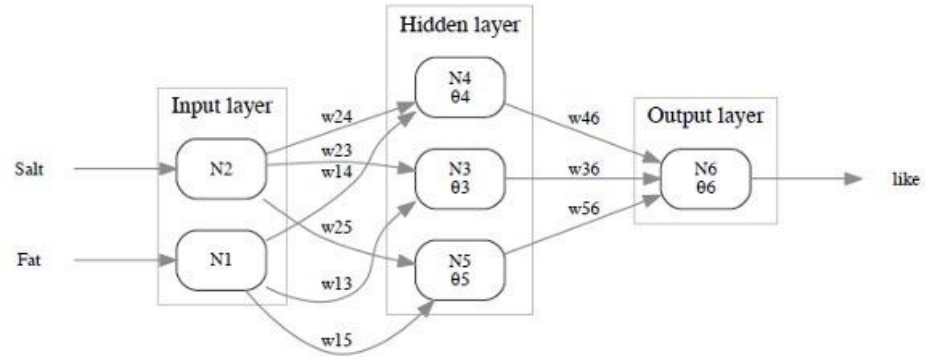
- Basic Idea
  - Combine input information in a complex & flexible neural network "model"
  - Model "coefficients" are continually tweaked **computing gradients** and then **backpropagating** to all weights
  - This is an iterative process. More epochs/iterations get you closer to a minimum error.

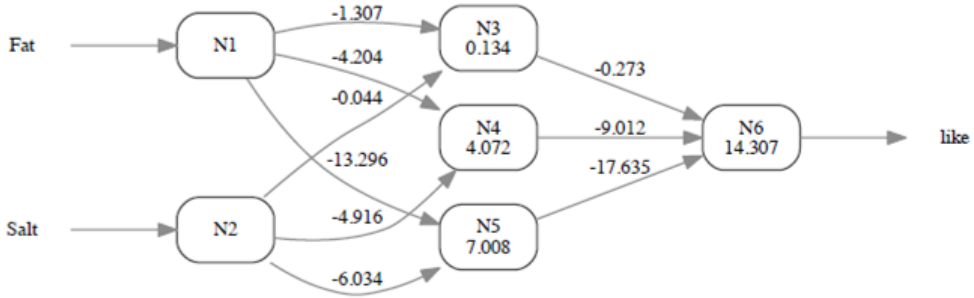


Example: predicting like/dislike, DMBA book 11.3

- Predict consumer opinion of a consumable product based on fat and salt content
- Rectangles are nodes, w_{ij} on arrows are weights, and Θ_j are node bias values
- The Input Layer
  - For record #1:
    - Fat input = output = 0.2
    - Salt input = output = 0.9
- The Hidden Layer
  - Has 3 nodes

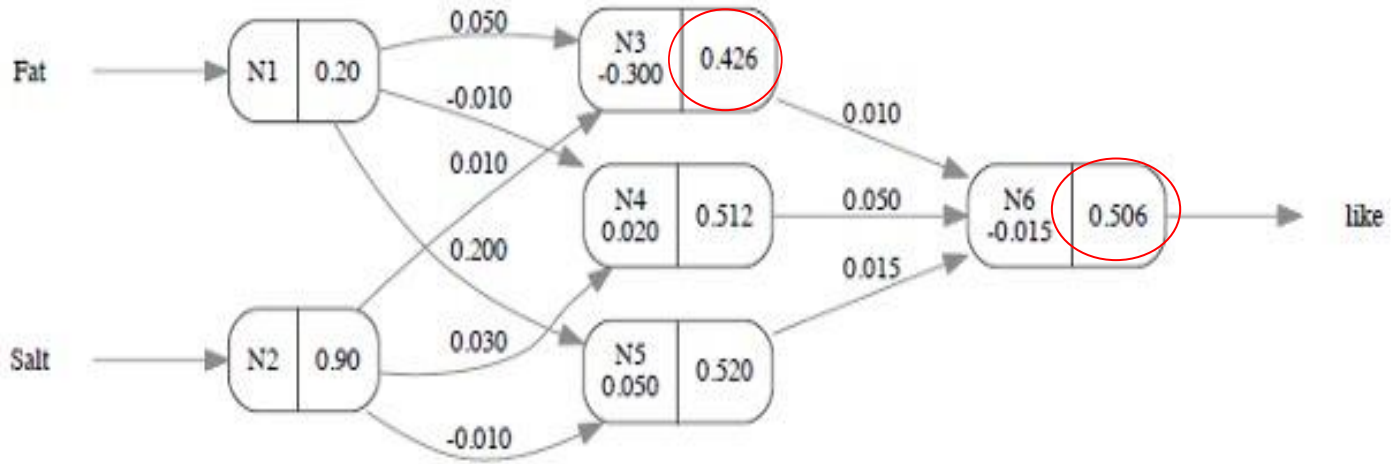
Obs.	Fat Score	Salt Score	Opinion
1	0.2	0.9	like
2	0.1	0.1	dislike
3	0.2	0.4	dislike
4	0.2	0.5	dislike
5	0.4	0.5	like
6	0.3	0.8	like





Example: Moving Through the Network

$$\text{Output}_{N3} = \frac{1}{1 + e^{-[-0.3+(0.05)(0.2)+(0.01)(0.9)]}} = 0.43$$



$$\text{Output}_{N6} = \frac{1}{1 + e^{-[-0.015 + (0.01)(0.43) + (0.05)(0.51) + (0.015)(0.52)]}} = 0.506.$$

Training the network is challenging!

- Preprocessing steps
  - All features are required to be numerical
  - Scale variables down, min-max, or standardization
    - Decide what is appropriate based on the nature of the activation function!
  - Categorical variables must be dummy coded
- **Iterative** passes through network
  - Goal: Find weights that yield best predictions
  - At each iteration compare predictions to actual values
    - Various types of loss functions (like RMSE) can be used to measure error
    - Error is propagated back and distributed to all the hidden nodes and used to update their weights
    - *l* reflects the "learning rate"
      - Too small: convergence extremely slow
      - Too large: faster learning, but may "overshoot" and fail to converge
- Mapping the output: E.g., in classification, output is between 0 and 1
  - Classification: use sigmoid
  - Prediction with metric output: Make sure last layer has no non-linearity.

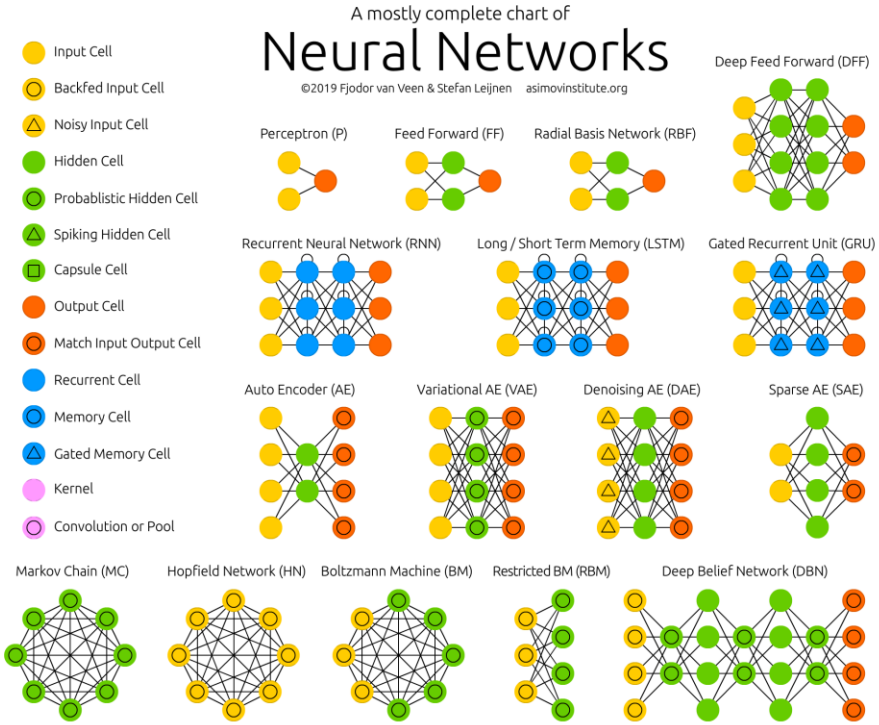
$$\theta_j^{new} = \theta_j^{old} + l(err_j)$$

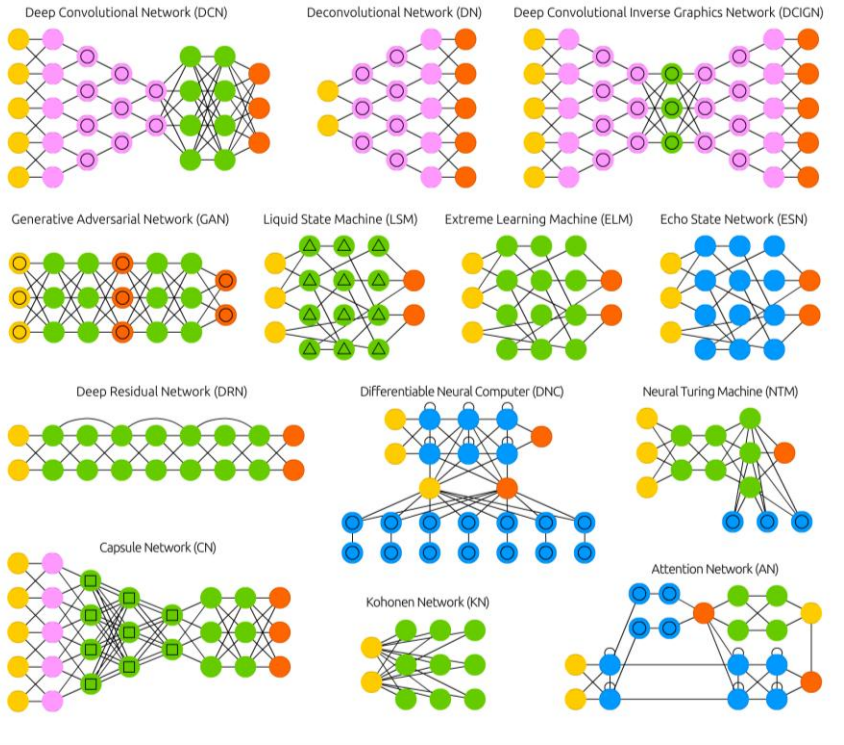
$$w_j^{new} = w_j^{old} + l(err_j)$$

Network Architecture Considerations

- Number of hidden layers: One hidden layer generally works well for simple datasets.
- Number of nodes in hidden layers: More nodes capture complexity, but increase chances of overfit
- Number of output nodes
  - For classification with m classes, use m or m-1 nodes
  - For numerical prediction use one
- Learning Rate
  - Low values lead to slow convergence. High values lead to instability
- With sufficient iterations, neural net can easily overfit the data. To avoid overfitting
  - Track error in validation data or via cross-validation
  - Limit iterations and/or complexity of network

Neural Network Architectures in practice





Demos

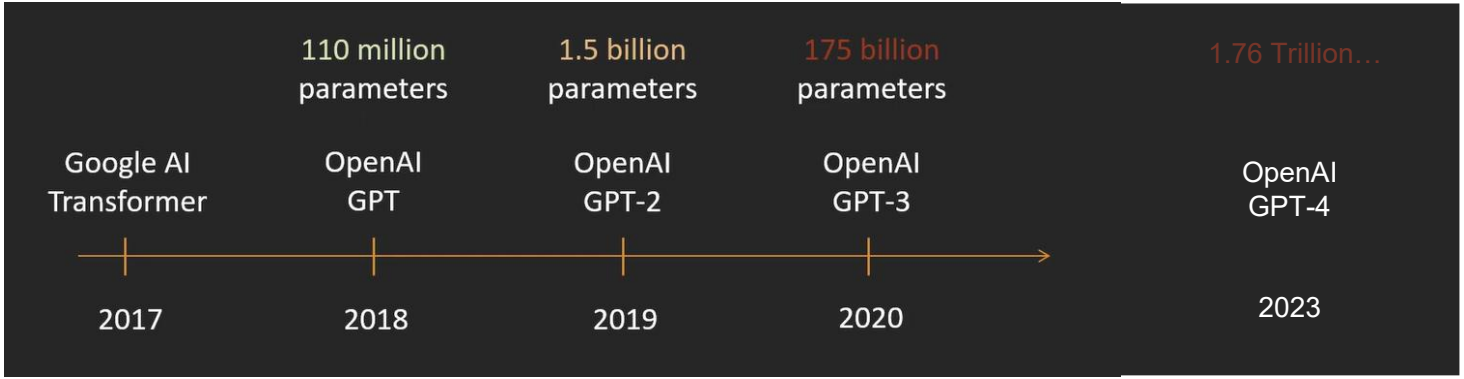
- Demo: Perceptron
- Demo 2: Neural Net
- Demo 3: Regression

Summary

- Advantages
  - Good predictive ability
  - Can capture flexible/complicated relationships between outcome and predictors

- Disadvantages
  - Major danger: tendency to overfit
  - A “black box” prediction machine, with no insight into relationships between predictors and outcome
  - Infinite architectures to choose from…
  - Requires large amounts of data
  - Heavy computational requirements if there are many variables (additional variables dramatically increase the number of weights to calculate)

Bonus: Deep Learning Models Are LARGE!!!



AI Coffee Beak with Letitia

Book Corpus
WebText

1,038 books (around 74M sentences and 1G words) of 16 different sub-genres (e.g., Romance, Historical, Adventure, etc.)

Common Crawl + …

Over 240 billion pages. Petabytes of data.

???
?

How much training does it take?

2 example models

GPT-3 (2020)

50,257 vocabulary size
2048 context length
175B parameters
Trained on 300B tokens

Model Name	n_{params}	n_{layers}	d_{model}	n_{heads}	d_{head}	Batch Size	Learning Rate
GPT-3 Small	125M	12	768	12	64	0.5M	6.0×10^{-4}
GPT-3 Medium	350M	24	1024	16	64	0.5M	3.0×10^{-4}
GPT-3 Large	760M	24	1536	16	96	0.5M	2.5×10^{-4}
GPT-3 XL	1.3B	24	2048	24	128	1M	2.0×10^{-4}
GPT-3 2.7B	2.7B	32	2560	32	80	1M	1.6×10^{-4}
GPT-3 6.7B	6.7B	32	4096	32	128	2M	1.2×10^{-4}
GPT-3 13B	13.0B	40	5140	40	128	2M	1.0×10^{-4}
GPT-3 175B or "GPT-3"	175.0B	96	12288	96	128	3.2M	0.6×10^{-4}

Table 2.1: Sizes, architectures, and learning hyper-parameters (batch size in tokens and learning rate) of the models which we trained. All models were trained for a total of 300 billion tokens.

Training: (rough order of magnitude to have in mind)

- O(1,000 - 10,000) V100 GPUs
- O(1) month of training
- O(1-10) $M

LLaMA (2023)

32,000 vocabulary size
2048 context length
65B parameters
Trained on 1-1.4T tokens

params	dimension	n heads	n layers	learning rate	batch size	n tokens
6.7B	4096	32	32	$3.0e^{-4}$	4M	1.0T
13.0B	5120	40	40	$3.0e^{-4}$	4M	1.0T
32.5B	6656	52	60	$1.5e^{-4}$	4M	1.4T
65.2B	8192	64	80	$1.5e^{-4}$	4M	1.4T

Table 2: **Model sizes, architectures, and optimization hyper-parameters.**

Training for 65B model:

- 2,048 A100 GPUs
- 21 days of training
- $5M

[Language Models are Few-Shot Learners, OpenAI 2020]
[LLaMA: Open and Efficient Foundation Language Models, Meta AI 2023]